Views and URL Routing in Django

MVC, View, URL

Simone Ricci

Model-View-Controller

Basics of a MVC design

Model-View-Controller (MVC)

Model-View-Controller is a software **architecture pattern**.

An architecture pattern is a general, reusable solution to a common problem that arises in software design.

Trygve Reenskaug created MVC while working on Smalltalk-79 in the late **1970s**.

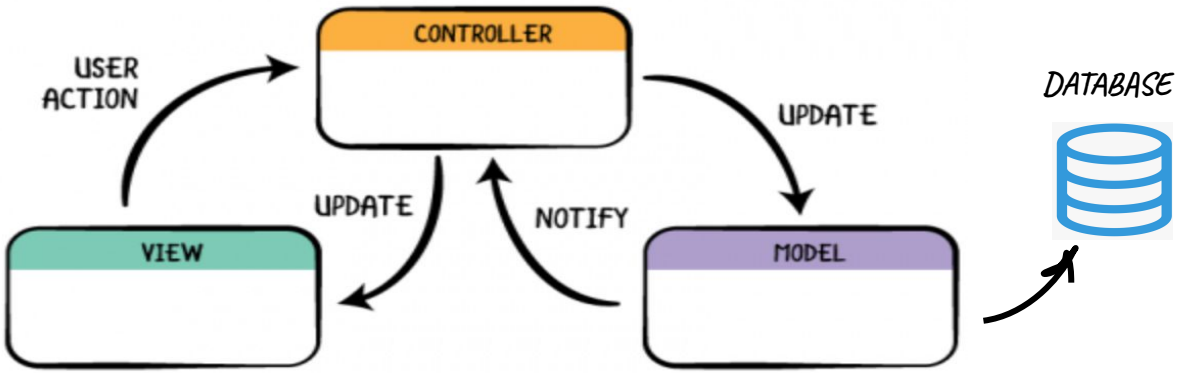
These types of pattern provide a set of guidelines for organizing the various components of a software system and defining their relationships to each other.

MVC is commonly used for developing **user interfaces** that divide the related program logic into **three** interconnected elements:

- **Model**
- **View**
- **Controller**

Model-View-Controller (MVC)

The MVC pattern helps to promote **separation of concerns and maintainability** of code.



- **Model**: it is application's data and business logic. The internal representations of data. (previous lesson..)
- **View**: it renders the data and provides a user interface. It **presents the data** from the model and **captures user input** for further processing.
- **Controller**: It acts as an **intermediary** between the model and the view. It receives input from the user via the view and uses that input to **manipulate the data** in the model. It then **updates the view** to reflect any changes in the data.

Django Model-View-Template (MVT)

Model-View-Template (MVT) pattern is a variation of the traditional MVC pattern.

The concept of the **template layer** is introduced to separate the user interface from the view logic.

- **Model**: Python classes that define the database schema and the associated data.
- **View**: Python functions or classes that handle HTTP requests and return HTTP responses. Views can also perform additional processing, such as authentication and authorization.
- **Template**: Text files that define the structure and presentation of the user interface. Templates are written in **HTML** with embedded **Django template language**.

In reality, Django's approach is a **4-part pattern** that also incorporates URL Configuration.

- **URL Configuration**: Regular-expression components configured to a View

Request-Response in Django

1) When a user type in a URL, such as https://djangoproject.com, the first thing that happens within a Django project is a search for a **URL pattern** (contained in **urls.py**).

2) The URL pattern matched is **linked to a single View** (contained in **views.py**).

3) The View combines the data from the **Model** (stored in **models.py**) and the styling from a **Template** (a file ***.html**).

4) The View then returns a **HTTP Response** to the client.

Django request/response cycle

```
HTTP Request -> URL -> View -> Model and Template -> HTTP Response
```

Complete Request-Response Cycle

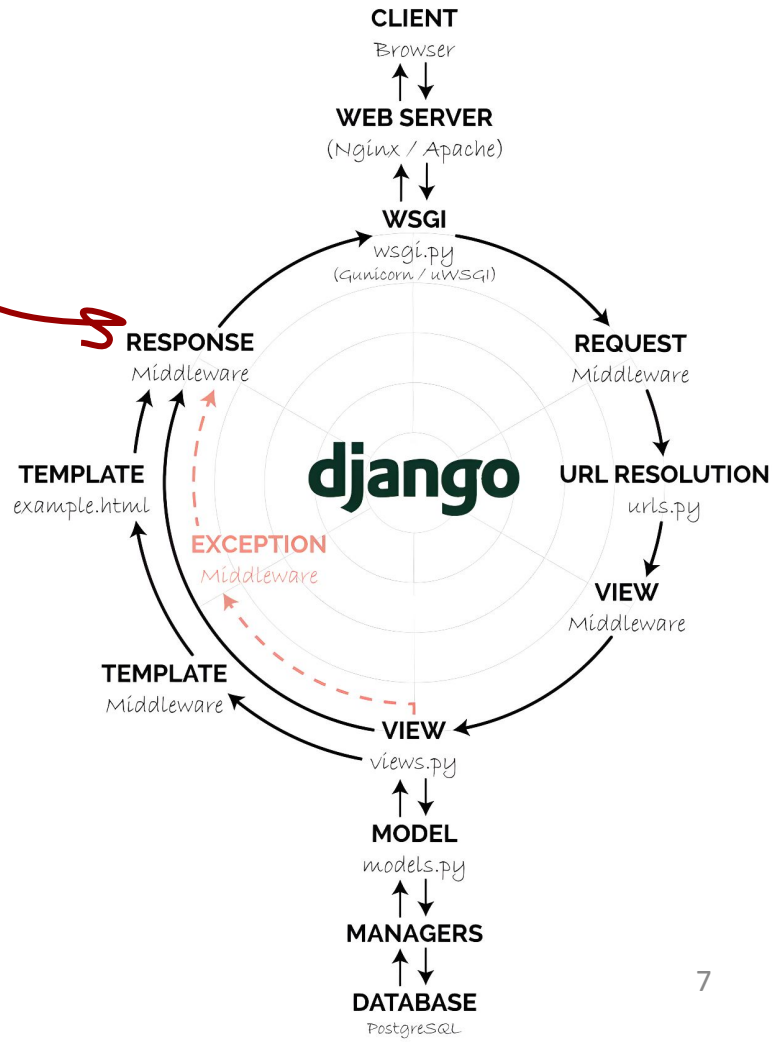
A Django Middleware is a framework of **hooks** into Django's **request/response processing**.

It's a light, low-level "plugin" system for globally altering Django's input or output.

Each middleware component is responsible for doing some **specific function**.

For example, **AuthenticationMiddleware** associates users with requests using sessions.

```
1 MIDDLEWARE = [
2     "django.middleware.security.SecurityMiddleware",
3     "django.contrib.sessions.middleware.SessionMiddleware",
4     "django.middleware.common.CommonMiddleware",
5     "django.middleware.csrf.CsrfViewMiddleware",
6     "django.contrib.auth.middleware.AuthenticationMiddleware",
7     "django.contrib.messages.middleware.MessageMiddleware",
8     "django.middleware.clickjacking.XFrameOptionsMiddleware",
9 ]
```



Django Middleware Uses Case

Uses for Middleware

- **Filtering**: filter out invalid or potentially malicious requests and have them return immediately (usually with an error response).
  - CSRF Middleware: checks if the request comes from a **valid origin** and immediately returns with a 403 Forbidden response if not.
  - Geo-blocking Middleware: filters out requests from certain geographical locations.
- **Injecting Data into Requests:** for example Django's Authentication Middleware, which adds a **user** object to every valid request. In the View and other middleware **request.user** can be used to access details of the logged in user.
- **Performing Logging, Analytics and Other Miscellaneous Tasks**

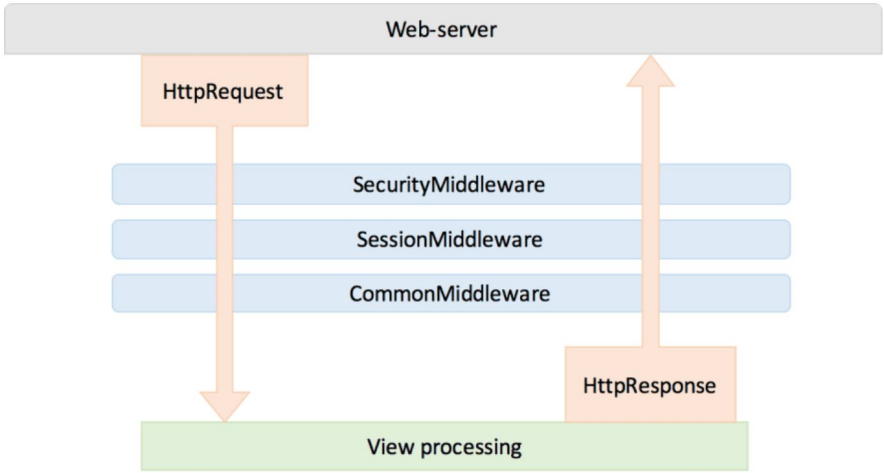
A Middleware is always global and it is applied to every request entering the system.

Django Middleware

Django applies middleware in the order it's defined in **MIDDLEWARE** (**settings.py**), top-down.

Onion structure: each middleware class is a "layer" that **wraps the View**, which is in the core of the onion.

If the request passes through all the layers of the onion (each one calls **get_response** to pass the request in to the next layer), all the way to the View, the response will then pass through every layer (in reverse order) on the way back out.



The response will only return through the same layers that the request passed in through.

Django Middleware Example

```python
class ExampleMiddleware:

    def _init_(self, get_response):
        # get_response is passed to our middleware by the Django framework.
        # It passes the request object to the next middleware, and gets the value of the response.
        self.get_response = get_response

    """
    Actual middleware logic. It is called by the Django framework to invoke our middleware.
    """
    def _call_(self, request):

        # Code that is executed in each request before the view is called

        response = self.get_response(request)

        # Code that is executed in each request after the view is called
        return response

    """
    We have access to the view function that Django is routing the request to
    along with any further arguments (args and kwargs) that will be passed to it
        """
    def process_view(self, request, view_func, view_args, view_kwargs):
        # This code is executed just before the view is called

    def process_exception(self, request, exception):
        # This code is executed if an exception is raised

    """
    This method is also invoked after the view has finished executing.
    It is only called if the resultant response contains a render() method,
    which indicates a template is being rendered.
    You can use this method to alter the content of the template,
    including its context data, if required.
    """
    def process_template_response(self, request, response):
        # This code is executed if the response contains a render() method
        return response
```

Before the View

After the View

Django Views

Views

A **View** is a **Python function (or class)** that takes a Request and returns a Response.

Views are responsible for processing and handling user requests to the web application.

They handle things like authentication, validation of user input, database queries, and the generation of dynamic HTML content.

Views are **typically** defined in Django apps using the **views.py** file.

In this file, you define functions that **handle specific user requests**: View that displays a list of products, and another View that allows users to create new products.

The **request object** contains information about the current request, such as the **HTTP Method** used (GET, POST, etc.), any **query parameters** passed in the URL, and any **form data** submitted in the request.

The **response object** contains the content that is returned to the user, such as HTML, JSON, 404 error, an XML document, a redirect …

Simple view

This is a function-based view that returns an **HttpResponse object** containing the string "Hello, world!"

```
from django.http import HttpResponse

def my_view(request):
    return HttpResponse("Hello, world!")
```

The **request** object passed to the view function ***my_view*** is an **HttpRequest object** that contains metadata about the request.

The HttpRequest object is created by Django when a web page is requested.

In the module **django.http** these two classes are defined:

```
def my_view(request):
    # ...
    if foo:
        return HttpResponseNotFound("<h1>Page not found</h1>")
    else:
        return HttpResponse("<h1>Page was found</h1>")
```

HttpRequest Class

All attributes should be considered **read-only!** (unless stated otherwise)

```
if request.method == "GET":
    do_something()
elif request.method == "POST":
    do_something_else()
```

Some of them are set by middleware:

- **method**: string representing the HTTP method used for the request, such as "GET", "POST", "PUT", "DELETE", etc.
- **GET**: dictionary-like object containing all the GET parameters of the request.
- **POST**: dictionary-like object with the POST parameters of the request.
- **path**: string containing the path of the requested URL, *without* the domain or query parameters.
- **META**: containing all available HTTP headers sent with the request, such as "HTTP_USER_AGENT".
- **session**: provides access to the session object for the current request.
- **user**: An attribute that represents the user making the request. It is an instance of the django.contrib.auth.models.User model, or an AnonymousUser object if the user *is not authenticated*.

HttpRequest methods

- **.get_full_path()**: returns the full path of the requested URL, including query parameters
- **.get_host()**: returns the HTTP host, as a string
- **.get_port()**: returns the port number used for the request, as an integer
- **.is_secure()**: returns True if the request is secure (if it was made with HTTPS)

Some of the method are alternative to attribute of the **HttpRequest** class.

QueryDict

In an HttpRequest object, the **GET** and **POST** attributes are instances of **Django.http.QueryDict**

The QueryDict class is used to **handle HTTP GET and POST data**.

It is a dictionary-like object that **can store multiple values for the same key**.

When a Django view receives an HTTP request, the QueryDict object is created to store the data from the request.

You can access this data in your view by accessing the **request.GET** or **request.POST** attribute, depending on the HTTP method used for the request.

QueryDict object will be **immutable** when accessed in a normal request/response cycle. To get a mutable version you need to use **QueryDict.copy()**.

QueryDict implements all the standard dictionary methods because it's a subclass of **Dictionary**.

QueryDict methods

.__init__(query_string=None, mutable=False, encoding=None): Constructor of the class

```
>>> QueryDict("a=1&a=2&c=3")
<QueryDict: {'a': ['1', '2'], 'c': ['3']}>
```

.get(key, default=None): returns the value for the given key. If the key has more than one value, it returns the ***last value***. Return a default value if the key doesn't exist.

.setdefault(key, default=None)

.update(other_dict): it appends (not replaces!!) to the current dictionary another QueryDict.

.getlist(key, default=None): Returns a list of the data with the requested key. Returns an empty list if the key doesn't exist and default is None

```
>>> q = QueryDict("a=1", mutable=True)
>>> q.update({"a": "2"})
>>> q.getlist("a")
['1', '2']
>>> q["a"]  # returns the last
'2'
```

HttpResponse class

Each View is responsible for instantiating, populating, and returning an **HttpResponse.**

Not like HttpRequest objects that are automatically generated by Django.

Typical usage is to pass the contents of the page, as a string, bytestring, or memoryview, to the **HttpResponse constructor**.

.__init__(content=b'', content_type=None, status=200, reason=None, charset=None, headers=None)

- **content_type:** MIME type optionally completed by a character set encoding and is used to fill the HTTP Content-Type header.
- **status:** HTTP status code for the response (use Python's http.HTTPStatus)
- **reason:** HTTP response phrase. If not provided, a default phrase will be used.
- **charset:** charset in which the response will be encoded.
- **headers:** dict of HTTP headers for the response.

```
>>> from django.http import HttpResponse
>>> response = HttpResponse("Here's the text of the web page.")
>>> response = HttpResponse("Text only, please.", content_type="text/plain")
>>> response = HttpResponse(b"Bytestrings are also accepted.")
>>> response = HttpResponse(memoryview(b"Memoryview as well."))
```

HttpResponse class

But if you want to add content incrementally, you can use response as a file-like object:

```
>>> response = HttpResponse()
>>> response.write("<p>Here's the text of the web page.</p>")
>>> response.write("<p>Here's another paragraph.</p>")
```

To set or remove a header field in your response, use HttpResponse.headers:

```
>>> response = HttpResponse()
>>> response.headers["Age"] = 120
>>> del response.headers["Age"]
```

You can also manipulate headers by treating your response like a dictionary. This proxies to HttpResponse.headers, and is the original interface offered by HttpResponse.

When using this interface, unlike a dictionary, del doesn't raise KeyError if the header field doesn't exist.

Class-based Generic Views

Early versions of Django only shipped with **function-based views**.

Developers found themselves **repeating the same patterns** over and over again: write a view that lists all objects in a model; write a view that displays only one detailed item from a model; …

Function-based generic views were introduced but were difficult to extend or customize.

For these reason Django introduced **class-based generic views**.

At its core, a **class-based view** allows you to respond to different HTTP request methods with **different class instance methods**, instead of with conditionally branching code inside a single view function.

```python
from django.http import HttpResponse

def my_view(request):
    if request.method == 'GET':
        # <view logic>
        return HttpResponse('result')
```

Function-based

```python
from django.http import HttpResponse
from django.views import View

class MyView(View):
    def get(self, request):
        # <view logic>
        return HttpResponse('result')
```

Class-based

Class to function View

Django's URL resolver expects to send the request and associated arguments to a **callable function**.

Class-based views have an **as_view()** class method which returns a function that can be called when a request arrives for a URL matching the associated pattern.

The function creates an instance of the class, calls **setup()** to initialize its attributes, and then calls its **dispatch()** method.

dispatch() looks at the request to determine whether it is a GET, POST, etc, and relays the request to a matching method if one is defined(**get()**, **post()**,..), or raises **HttpResponseNotAllowed** if not.

Mixins

In object-oriented programming, **inheritance** is a mechanism that allows a subclass to inherit properties and behavior from its superclass.

A **Mixin** is essentially a class that defines a set of methods that can be added to other classes.

Mixin is implemented as a class that defines a set of methods, but **doesn't define any state** (i.e., instance variables).

The methods in the mixin can access and modify the state of the target class, but the mixin itself **doesn't have any state**.

In Django it's defined a mixin called **TemplateResponseMixin** whose primary purpose is to define the method **render_to_response()**.

When combined with the View base class, the result is a **TemplateView class** that has the behavior defined in the View base class, and the **render_to_response()** method.

Inerithance Example

Use generic Views is to inherit from an existing view and **override attributes** (such as the **template_name**) **or methods** (such as **get_context_data**) in your subclass to provide new values or methods.

Consider, for example, a view that just displays the template written in **about.html**.

Django has a generic view to do this: **TemplateView**. We can subclass it, and override the template name:

```
# some_app/views.py
from django.views.generic import TemplateView


class AboutView(TemplateView):
    template_name = "about.html"

```

Function-based Views and Class-based Views.

Function-based views are simple Python functions that take a **request object** as their argument and return an **HTTP response**.

They are the most common type of view in Django and are very flexible, making them a good choice for small and medium-sized projects.

Class-based views, are defined as Python classes that inherit from one of Django's view classes (such as **View**, **TemplateView**, **ListView**, **DetailView**, etc.).

They provide a way to encapsulate related behavior into a single object and handle HTTP requests using class methods (get(), post(), put(), delete(), etc.) which allow to handle easily different HTTP methods using the same view class.

Class-based views provide **reusable functionality through inheritance**, making it easier to build a hierarchy of views that share common functionality.

Class-based views provide **built-in support for common patterns**, such as displaying a list of objects (**ListView**), displaying a detail view of an object (**DetailView**), displaying a form (**FormView**), etc.

Django URL

URL in Django

URLs in Django **map incoming requests** from a client's browser to the corresponding view functions that will generate the HTTP response.

In Django, URLs are defined using **URL patterns**. They are defined in the **urls.py** file of each Django application.

A clean, elegant URL scheme is an important detail in a high-quality web application.

See Cool URIs don't change, by World Wide Web creator Tim Berners-Lee, for excellent arguments on why **URLs should be clean and usable**.

> What makes a cool URI?
> A cool URI is one which does not change.
> What sorts of URI change?
> *URIs don't change: people change them.*

URLs flow:

1. The **Django's URL dispatcher** (which is part of the core framework) receives the request and examines the URL.
2. The URL dispatcher consults the URLconf module (**urls.py**) to find a match for the requested URL.
3. If a match is found, the URL dispatcher calls the corresponding view function, passing any captured URL parameters as arguments.

URL dispatcher

Django determines the **root URLconf** module to use. This is the value of the **ROOT_URLCONF** in **setting.py**.

1) Django loads that Python module and looks for the variable **urlpatterns**. This should be a sequence of **django.urls.path()** and/or **django.urls.re_path()** instances.
2) Django runs through each URL pattern, **in order**, and **stops at the first** one that matches the requested URL, matching against **path_info** attribute of the **HttpRequest**.
3) Once one of the URL patterns matches, Django imports and calls the given view.
4) The view gets passed the instance of the HttpRequest plus some arguments.

If no URL pattern matches, or if an exception is raised during any point in this process, Django invokes an appropriate **error-handling view.**

urlpatterns

Parameters are passed to the corresponding view function, allowing it to generate a **dynamic response** based on the request.

To capture a value from the URL we use angle brackets **<>**.

Captured values can optionally include a **converter type**. For example, use **<int:name>** to capture an integer parameter.

There's no need to add a leading slash, because every URL has that. For example, it's "articles", not "/articles".

```
from django.urls import path

from . import views

urlpatterns = [
    path("articles/2003/", views.special_case_2003),
    path("articles/<int:year>/", views.year_archive),
    path("articles/<int:year>/<int:month>/", views.month_archive),
    path("articles/<int:year>/<int:month>/<slug:slug>/", views.article_detail),
]
```

/articles/2005/03/ */articles/2003/* */articles/2003*
/articles/2003/03/building-a-django-site/

Path converters

The following path converters are available by default:

- **str** - Matches any non-empty string, excluding the path separator, '/'. This is the default if a converter isn't included in the expression.
- **int** - Matches zero or any positive integer. Returns an int.
- **slug** - Matches any slug string consisting of ASCII letters or numbers, plus the hyphen and underscore characters. For example, *building-your-1st-django-site*.
- **uuid** - Matches a formatted UUID. To prevent multiple URLs from mapping to the same page, **dashes must be included and letters must be lowercase**. For example, 075194d3-6885-417e-a8a8-6c931e272f00. Returns a UUID instance.
- **path** - Matches any non-empty string, including the path separator, '/'. This allows you to match against a complete URL path rather than a segment of a URL path as with str.

Defaults arguments for view

The URLconf searches against the requested URL, as a normal Python string. This does **not include** GET or POST parameters, or the domain name.

For example, in a request to https://www.example.com/myapp/, the URLconf will look for *myapp/*

In a request to https://www.example.com/myapp/?page=3, the URLconf will look for *myapp/*

All request methods – POST, GET, etc. – will be routed to the same function for the same URL. A convenient trick is to **specify default parameters** for views' arguments.

```
urlpatterns = [
    path("blog/", views.page),
    path("blog/page<int:num>/", views.page),
]


# View (in blog/views.py)
def page(request, num=1):
    # Output the appropriate page of blog entries, according to num.
    ...
```

Including other URLconfs

urlpatterns can "**include**" other URLconf modules.

This essentially "roots" a set of URLs below other ones.

Whenever Django encounters **include(),** it chops off whatever part of the URL matched up to that point and sends the remaining string to the included URLconf for further processing.

An included URLconf **receives any captured parameters from parent URLconfs**!

```
from django.urls import include, path

urlpatterns = [
    # ... snip ...
    path("community/", include("aggregator.urls")),
    path("contact/", include("contact.urls")),
    # ... snip ...
]
```

Passing extra options to view

URLconfs have a hook that lets you pass **extra arguments** to your view functions, as a Python **dictionary**.

The **path()** function can take an optional third argument which should be a dictionary of extra keyword arguments to pass to the view function.

```
from django.urls import path
from . import views

urlpatterns = [
    path("blog/<int:year>/", views.year_archive, {"foo": "bar"}),
]
```

For a request to */blog/2005/*, Django will call *views.year_archive(request, year=2005, foo='bar')*.

Extra options can be passed to **include()** and **each line in the included URLconf** will be passed the extra options.

You must ensure that each view of the included URLconf accepts the extra options!

Reverse resolution of URLs

Obtain URLs in their final forms for generated content (Views, URLs shown to the user, etc.) or for handling of the navigation flow (redirections, etc.) is essential.

Try to **avoid hard-coding** these URLs (**non-scalable** and **error-prone** strategy).

Django offers a **DRY** (Don't repeat yourself) mechanism. It allow evolution of the URL design without having to search and replace outdated URLs. It needs:

- An **identification** (name) of the view in charge of handling it.
- Other pieces of information are the types (positional, keyword) and values of the view arguments.

Starting with the name of the corresponding Django view and the values of arguments, the URL mapping obtain the associated URL. This is **reverse resolution**.

- In templates: Using the ***url*** template tag.
- In Python code: Using the **reverse()** function.
- For handling URLs of Django model instances: The **get_absolute_url()** method.

Naming URL patterns

In order to perform **URL reversing**, you'll need to use **named URL patterns**.

Choose names that are **unlikely to clash** with other applications' choice of names.

If you call your URL pattern the same as another application does, the URL that **reverse()** finds depends on whichever pattern **is last** in your project's urlpatterns list.

```
urlpatterns = [
    # ...
    path("articles/<int:year>/", views.year_archive, name="news-year-archive"),
    # ...
]
```

```
from django.http import HttpResponseRedirect
from django.urls import reverse


def redirect_to_year(request):
    # ...
    year = 2006
    # ...
    return HttpResponseRedirect(reverse("news-year-archive", args=(year,)))
```

URL namespaces

URL namespaces allow you to uniquely reverse named URL patterns even if **different applications** use the same **URL names**. Namespaced URLs are specified using the '**:**' operator.

It's a good practice for third-party apps to always use namespaced URLs!

A URL namespace has two parts:

1) **application namespace**: describes the name of the application that is being deployed. For example, Django's admin application has the somewhat predictable application namespace of "admin"

2) **instance namespace**: identifies a specific instance of an application. Instance namespaces should be unique across your entire project.

For example, the main index page of the admin application is referenced using 'admin:index'. This indicates a namespace of 'admin', and a named URL of 'index'.

The named URL 'sports:polls:index' would look for a pattern named 'index' in the namespace 'polls' that is itself defined within the top-level namespace 'sports'.

URL namespaces

urls.py ¶

```
from django.urls import include, path

urlpatterns = [
    path("author-polls/", include("polls.urls", namespace="author-polls")),
    path("publisher-polls/", include("polls.urls", namespace="publisher-polls")),
]
```

polls/urls.py ¶

```
from django.urls import path

from . import views

app_name = "polls"
urlpatterns = [
    path("", views.IndexView.as_view(), name="index"),
    path("<int:pk>/", views.DetailView.as_view(), name="detail"),
    ...,
]
```

Assume we have **enhanced** polls application so that it takes the instance namespace into consideration when creating and displaying polls.

If one of the instances is current (we are rendering the detail page in the instance 'author-polls') 'polls:index' will resolve to the index page of the 'author-polls' instance; i.e. the following will result in "**/author-polls/**":

reverse("polls:index", current_app = self.request.resolver_match.namespace)

Article Project Live